

Formula 110 — Laboratory Notebook

Entry 1 — Reactive Controller Foundation (Strategy 1: Reactive Control + Parameter Optimization)

Date and time: August 30, 2026, evening session

Participants and contributions:

- Leon Aaron and Jayden Webb— designed the sensor-to-command control logic, wrote the controller and evaluation harness, ran and interpreted experiments, committed the result.
- AI coding agent (Claude Code) — read the simulator's sensor, physics, and scoring internals to g

Question or objective: Can a hand-written reactive controller (explicit sensor→command rules with tunable parameters) drive the car around the full track and complete at least one lap within 30 seconds, from a random seeded starting position, without being destroyed (damage reaching 1.0)? This is the foundation for both our baseline and the parameter-optimization step that follows.

What we investigated or changed:

- Read the simulator's public contract (`src/racing/student/api.py`, `SENSORS.md`) and internals (`race/sensors.py`, `race/progress.py`, `race/runtime.py`, `physics/engine.py`) to confirm sensor sign conventions, the lap-counting rule (cumulative forward progress since spawn crossing the track length — not a literal return to the start/finish line), and the damage model where car is eliminated at damage ≥ 1.0 .
- Measured track geometry directly: total closed-loop length ≈ 181.1 m, track width 6.6 m (~ 3.3 car-widths).
- Implemented `src/controllers/reactive.py` — a parametrized controller (`ReactiveParams` dataclass) that steers using centerline offset, heading error, and near/far lookahead offsets; adds a proximity-based wall-avoidance term; brakes proportionally to a speed-scaled stopping distance against `wall_lidar.front_m`; and falls back to an open-side recovery maneuver on any contact.
- Implemented `scripts/evaluate_controller.py` — a headless evaluation harness that races the controller against a passive baseline across seeded starting positions and reports scored distance, laps, damage, and marshal activity. This becomes the fitness function for the parameter-optimization stage.
- Ran the harness with a conservative speed target (`max_speed_mps = 9.5`); after observing zero damage, deliberately raised it (`14.0`) and loosened the cornering-slowdown threshold to spend the unused safety margin on speed.

Evidence:

- Sources or documentation: `SENSORS.md`, `README.md`, `src/racing/student/api.py`, `src/racing/race/sensors.py`, `src/racing/race/progress.py`, `src/racing/race/runtime.py`, `src/racing/race/head_to_head.py`, `src/racing/physics/engine.py`, `src/racing/track/world.py`.
- AI-agent assistance: Claude Code explored the simulator internals above to verify design assumptions before writing code, then implemented both files and ran the evaluation script itself, iterating parameters based on printed results rather than submitting a first guess.

- Commits or code: commit `cc38354`, "Start with reactive control and parameter optimization" — adds `src/controllers/reactive.py` and `scripts/evaluate_controller.py`.
- Experiment output: `uv run python scripts/evaluate_controller.py --module controllers.reactive --seed <seed> --races 5 --round-seconds 30`, run across the recommended seed suite (42, 110, 271, 997, 2027).
- Leaderboard result: not yet submitted — this stage used local headless evaluation only.

What we observed:

- First pass (`max_speed_mps = 9.5`): 5/5 races survived with zero damage, but only 2/5 crossed the ~181 m lap threshold (177–187 m raw distance) — safe but not reliably fast enough to guarantee a lap.
- After raising `max_speed_mps` to 14.0 and the cornering-slowdown trigger from 35° to 50° of heading error: 5/5 survived, 5/5 completed ≥ 1 lap (avg. 234.5 m in 30 s), still zero damage.
- Re-running across all five recommended seeds (25 races total): **25/25 survived, 25/25 completed ≥ 1 lap, damage = 0.00 in every race**, with no marshal resets triggered.
- The one notable "failure" was the first speed setting — safe but too conservative to guarantee a lap. The fix came from the measured data (laps completed vs. distance short of threshold), not a guess.

Decision and rationale: Adopted the `max_speed_mps = 14.0`, `corner_heading_error_deg = 50.0` configuration as the baseline reactive controller (`DEFAULT_PARAMS` in `src/controllers/reactive.py`). It satisfies the deliverable — survives and completes ≥ 1 lap within 30 s — with full damage margin (0.00 observed) across every tested seed, indicating room to push further in the optimization stage rather than trading speed for safety.

Next steps:

- Feed `scripts/evaluate_controller.py`'s output (avg. scored distance, penalized for any elimination) into an automated parameter search (Optuna or CMA-ES) over `ReactiveParams`.
- Widen evaluation to more seeds/races per trial before trusting an optimized parameter set, to avoid overfitting to the five recommended seeds.
- Begin the second experimental approach (neuroevolution over a small MLP) on the same evaluation harness, per the two-approach exploration requirement.

Entry 2 — Neuroevolution Foundation (Strategy 2: Evolution Strategy over a Small MLP)

Date and time: August 30, 2026, evening session (continued)

Participants and contributions:

- Leon Aaron and Jayden Webb — reviewed each training result, decided when to change the fitness function vs. accept a result, and directed the scope of the experiment.
- AI coding agent (Claude Code) — implemented the network, feature encoding, and evolution-strategy trainer; ran four training iterations; diagnosed each failure mode from the printed generation-by-generation fitness and held-out validation output; proposed and implemented each fitness-function fix.

Question or objective: Can a small neural-network controller (10 sensor inputs $\rightarrow$ 6-unit hidden layer $\rightarrow$ 2 outputs), with its weights evolved by a simple evolution strategy rather than hand-written, reach the same goal as Approach 1 — drive around the track and complete a lap from a random seeded start without being destroyed? This is the second, mechanistically distinct approach required by the exploration stage.

What we investigated or changed:

- Implemented `src/controllers/neuro.py` (network forward pass, sensor-to-feature encoding, `Genome` type) and `scripts/train_neuroevolution.py` (a dependency-free ($\mu + \lambda$) evolution strategy with elitism and Gaussian mutation), reusing `scripts/evaluate_controller.py` as the shared fitness/validation harness so results are directly comparable to Approach 1.
- Ran four training iterations, changing the fitness function each time in response to what the previous attempt's held-out validation revealed:
 - `distance - 300 if eliminated`, single training seed $\rightarrow$ training fitness 225 m, but held-out validation showed the genome crashing in most races (17/25 eliminated) after covering enough distance to still score well — the penalty was too weak relative to achievable high-speed distance.
 - Switched to a fixed penalty that fully replaces distance on elimination (survival-only credit) $\rightarrow$ training fitness dropped to 63 m, and validation showed the population had converged to standing nearly still (0/25 laps) — "never move" is a free, zero-risk optimum under a purely survival-dominant fitness.
 - Added a graduated fitness (distance - 350 if eliminated, distance - 20 if idle below 10 m, else raw distance), single training seed $\rightarrow$ training fitness 260 m; validation showed genuinely fast driving (28–35 m/s, 21/25 laps completed) but still frequent elimination (8/25 survived) — safety learned on one starting position did not generalize.
 - Same fitness as (3), trained on two seeds instead of one to force cross-seed safety $\rightarrow$ training fitness dropped to 53 m and reproduced attempt (2)'s idle genome exactly — the harder, multi-seed objective exceeded what a 10-genome, 8-generation search could solve.

Evidence:

- Sources or documentation: none beyond what was already read for Entry 1; this stage was empirical (iterate $\rightarrow$ measure $\rightarrow$ diagnose).
- AI-agent assistance: Claude Code wrote all four fitness-function versions and ran every training/validation pass; each change was motivated by inspecting the concrete failure in the previous attempt's output rather than a fresh guess.
- Commits or code: `src/controllers/neuro.py`, `scripts/train_neuroevolution.py` (not yet committed as of this entry).
- Experiment output: `uv run python scripts/train_neuroevolution.py [--population --generations --seeds --round-seconds]` for each attempt; `uv run python scripts/evaluate_controller.py --module controllers.neuro --seed <seed> --races 5 --round-seconds 30` across (42, 110, 271, 997, 2027) to validate each resulting genome.
- Leaderboard result: not applicable — local evaluation only.

What we observed:

- Every attempt that rewarded raw distance strongly enough to produce real driving also under-punished crashing relative to the distance a fast, reckless genome could rack up before crashing (attempts 1 and 3).
- Every attempt that punished elimination hard enough to suppress recklessness also made "not moving" a competitive or dominant strategy for this small a population/generation budget (attempts 2 and 4).
- Requiring safety across two training seeds simultaneously (attempt 4) was strictly harder for this search method than requiring it on one (attempt 3) — the search collapsed to the degenerate idle solution

instead.

- No attempt matched Approach 1's result (25/25 survived and lapped). The best population/generation budget tested (10–12 genomes, 8–10 generations) is genuinely too small to resolve the speed/safety trade-off with this simple mutation-only evolution strategy.

Decision and rationale: Kept attempt 3's genome as `BEST_GENOME` in `src/controllers/neuro.py` — it is the most informative result (demonstrates real racing behavior and a materially higher top speed than Approach 1, 28–35 m/s vs. 13.9 m/s) even though it is not yet reliable. Documented all four attempts rather than only the best one, since the failure modes themselves are the useful evidence for comparing the two approaches (see `EXPLORATION.md`).

Next steps:

- Increase the evolutionary search budget (larger population, more generations, or true CMA-ES with covariance adaptation) now that the fitness function's failure modes are understood.
- If neuroevolution is retained past the exploration stage, consider warm-starting the network by cloning Approach 1's reactive controller (behavior cloning) before evolving further, rather than starting from random weights.
- Proceed to the "Select and Refine" stage using Approach 1 as the primary controller, per the comparison in `EXPLORATION.md`.

Entry 3 — Reactive Controller Refinement: Apex-Hugging Steering + Parameter Optimization

Date and time: August 31, 2026

Participants and contributions:

- Leon Aaron — proposed the specific behavioral changes (no proactive braking, infer turn sharpness from sensors, hug the inside of a turn), reviewed each result, and directed the scope of the optimization run.
- AI coding agent (Claude Code) — implemented the apex-hugging steering and no-proactive-braking redesign, built the parameter-search script and its diagnostic tracing, ran the search, and validated the result.

Question or objective: Can the reactive controller from Entry 1 go substantially faster than its 229 m/30s baseline, using two concrete ideas: (1) inferring the sharpness of an upcoming turn from `camera.heading_error_degrees` and biasing the car toward the inside of that turn rather than the centerline, and (2) removing proactive cornering deceleration so throttle always targets top speed and only the reactive wall-proximity check can brake? And can a parameter search then push the resulting design further while an added diagnostic tool explains *why* a given parameter set does well or poorly?

What we investigated or changed:

- Redesigned `src/controllers/reactive.py`: added `_apex_target_offset_m`, which biases the steering "centering" target toward the inside of the turn (right side on a right turn, left side on a left turn) using `turn_severity = |heading_error_degrees| / turn_sharpness_deg`. Removed the old curvature-based speed target entirely — throttle now always targets `max_speed_mps`, with the wall-proximity brake as the only source of negative throttle.

- Key mechanical insight that justifies apex-hugging in this simulator specifically: scored distance is the car's projection onto the fixed centerline, not its physical path length, so cutting the inside of a turn advances scored distance per meter actually driven — the geometry argument holds for *this* scoring rule, not just as general racing-line folklore.
- Verified the redesign alone (default parameters, no search yet) against the full seed suite before doing anything else: **25/25 survived, 25/25 laps, 0.00 damage, avg. 259 m/30s** — already +13% over Entry 1's 229 m, from apex-cutting alone at the same top speed.
- Built `scripts/optimize_reactive.py`: a (mu + lambda) evolution strategy seeded from the known-good default parameters (not random init — an explicit lesson carried over from Entry 2's random-init struggles), with per-parameter mutation scaled to each parameter's own magnitude, and the same three-tier fitness (elimination penalty, idle penalty, else raw distance) that worked for the neuroevolution trainer.
- Added a `--diagnose` mode that reruns one race with full per-tick tracing (sensors, command, and simulated time) and prints: how often the car braked, how often a wall was close, the sensor/command state at the first contact, and the final ticks before an elimination — a concrete answer to "why did this do well or poorly", not just a fitness number.
- Ran `--diagnose` on the pre-search defaults first: revealed the wall-proximity brake was firing on **43% of ticks** — not from any cornering logic (already removed), but because its safety margin (`speed × 1.1s` lead time) scales up to ~15 m at top speed on a track this curvy, making the "reactive-only" brake accidentally proactive. This was a concrete, sensor-and-time-stamped finding, not a guess.
- Ran the search (population 10, generations 10, elite 3, training seeds 13 and 55 — distinct from the held-out suite, 20-second rounds): fitness rose from 189.8 m (generation 0) to 289.3 m (generation 9).

Evidence:

- Sources or documentation: none new; built on Entry 1's sensor/scoring findings.
- AI-agent assistance: Claude Code implemented the redesign, the search script, and the diagnostic tool; used the diagnostic tool's own output (the 43% braking finding) to decide the search should include the brake thresholds as free parameters rather than assuming a fixed value.
- Commits or code: `src/controllers/reactive.py` (redesigned + optimized `DEFAULT_PARAMS`), `scripts/optimize_reactive.py` (new).
- Experiment output: `uv run python scripts/optimize_reactive.py --population 10 --generations 10 --seeds 13 55 --round-seconds 20` for the search; `uv run python scripts/evaluate_controller.py --module controllers.reactive --seed <seed> --races 5 --round-seconds 30` across (42, 110, 271, 997, 2027) for held-out validation.
- Leaderboard result: not applicable — local evaluation only.

What we observed:

- The optimized parameters cut braking from 43% of ticks to about 1%, raised `max_speed_mps` from 14.0 to 15.8, and tightened the apex bias (`apex_bias_max_m` settled at 0.46 m with a much smaller `turn_sharpness_deg` of 13°, i.e. a sharper, earlier bias reaction than the hand-picked defaults).
- Held-out validation across all five recommended seeds (25 races): **25/25 survived, 25/25 completed 2 laps (not just 1), damage = 0.00 in every race**, average scored distance **≈435 m per 30 s** — roughly **90% more distance than Entry 1's original 229 m**, with an equally perfect safety record.
- No failed attempt to report this time — seeding the search from a known-good baseline (rather than random init, as neuroevolution was forced to use) meant every generation's worst genome was still a

plausible driver, not a catastrophic one; the search converged smoothly without the degenerate local optima seen in Entry 2.

Decision and rationale: Adopted the optimized parameter set as the new `DEFAULT_PARAMS`. The evidence (25/25 survival and lap completion at every tested seed, zero damage, and a large, real speed increase) clearly supports keeping it as the primary controller, and confirms the apex-hugging/no-proactive-braking hypothesis: the improvement came from a combination of the redesigned control law (+13% before any search) and the parameter search built on top of it (+68% more on top of that).

Next steps:

- Try a larger search budget (more generations/population) now that a single search run took under 3 minutes wall-clock, to see if further gains are available.
- Investigate whether `steer_limit` settling at 0.63 (well below 1.0) indicates the car is leaving speed on the table in the sharpest turns; consider whether a track-position-aware `apex_bias_max_m` (larger in sharper, slower corners) would help.
- This result is now the baseline to beat for the "Select and Refine" stage; compare any future controller (including a re-attempted, better-budgeted neuroevolution run) against this ~435 m figure, not the original 229 m.

Entry 4 — Failed Attempt: Anticipatory Apex Bias + Higher-Speed Search (Reverted)

Date and time: August 31, 2026

Participants and contributions:

- Leon Aaron — requested the specific behavior (drive straight through small chicanes, hug the inside of turns harder, target ~2x distance) and, after reviewing the risky validation numbers, called for reverting to the known-good Entry 3 state and asked for manual tuning guidance instead of further automated search.
- AI coding agent (Claude Code) — implemented the redesign, ran two search attempts, diagnosed why each was unsafe, and performed the revert.

Question or objective: Could the reactive controller reach roughly double Entry 3's distance (~435 m → ~870 m) by (1) making the apex-bias steering anticipatory — driven by the farthest camera lookahead point (~16 m ahead) instead of instantaneous heading error, so a brief chicane nets out straight while a sustained turn still gets the full inside-hugging bias — and (2) searching a wider parameter range, including a much higher `max_speed_mps`?

What we investigated or changed:

- Verified the track geometry directly (segment headings from `MUGELLO_SHORT_LAYOUT`): confirmed a ~49-unit straight at the start and exactly two sharp right-hand turns (~50–52°, near "Luco climb" and "Materassi exit"), each in an otherwise left-trending section — consistent with what was reported by observation.
- Replaced the heading-error-driven apex bias with a lookahead-driven one (`lookahead_offsets_m[-1]`), and manually raised `max_speed_mps` to 20.0 and `apex_bias_max_m` to 1.2 as a search starting point.

- Ran a parameter search (population 12, generations 14, seeds 13 & 55, 20-second rounds): training fitness reached 314 m, but the trace showed **real, non-zero damage (0.27–0.41) on both training seeds** — the fitness function only penalized full elimination (damage ≥ 1.0), never partial damage, so it rewarded a genome that clips walls hard but happens not to fully break in a short training round.
- Validated that genome on the full held-out suite anyway: **10/25 survived**, with damage = 1.0 (full elimination) in most of the losses — confirming the fitness gap was real, not a fluke.
- Fixed the fitness function (`scripts/optimize_reactive.py`) to subtract a penalty proportional to non-fatal damage, not just elimination, and reran the search with the same budget.
- The corrected search converged to `max_speed_mps  $\approx 19.1$` with small training damage (0.06–0.11). Validated on the held-out suite: **19/25 survived, average distance 399.7 m** — safer than the first attempt, but still worse on both counts than Entry 3's 25/25-survived, 435 m result.

Evidence:

- AI-agent assistance: Claude Code ran both search attempts in the background, diagnosed the fitness-function gap from the trace output (not a guess — the 0.27–0.41 training damage was visible before validation confirmed the consequence), and reverted the shipped controller only after the second attempt also underperformed the known-good baseline.
- Commits or code: `scripts/optimize_reactive.py` retains the damage-penalty fix (a genuine correctness improvement, kept); `src/controllers/reactive.py` was reverted to the exact Entry 3 state and re-verified.
- Experiment output: two full background search runs (population 12, generations 14, seeds 13 & 55, 20s rounds; ~13–15 min wall-clock each) plus held-out validation runs (5 seeds $\times$ 5 races $\times$ 30s) after each.
- Leaderboard result: not applicable.

What we observed:

- Both search attempts found genomes that looked good on their two training seeds but generalized worse than the existing baseline on the held-out suite — the same "training seed(s) don't cover the whole risk surface" failure pattern seen with neuroevolution in Entry 2, now reproduced in the reactive-controller search too.
- Fixing the fitness function (penalizing damage, not just elimination) measurably reduced training-time recklessness (0.27–0.41 damage $\rightarrow$ 0.06–0.11 damage) but was still not enough alone to beat the existing 435 m/0-damage baseline within this search budget.
- Average distance is not a safe stand-in for "twice as fast without being destroyed": both search attempts had *higher peak per-race distances* (up to 456 m) than Entry 3, but lower *average* distance once eliminations are included, and violated the hard "not destroyed" requirement on a meaningful fraction of races.

Decision and rationale: Reverted `src/controllers/reactive.py` to the exact Entry 3 parameter set (25/25 survived, 0.00 damage, ~435 m) rather than keep either riskier result. Two training seeds with a 20-second round appears to be too small a sample of this track's risk surface for this search budget to reliably find a genuinely safer-and-faster optimum — the search kept discovering the same trade-off (more speed, more wall contact) without enough pressure to escape it. Manual, incremental tuning — changing one parameter at a time and validating against all 5 held-out seeds after each change — is the next step, so a human can catch a "looks great on training seeds, fails on held-out seeds" result immediately rather than trusting one aggregate fitness number.

Next steps:

- Manually probe the safe ceiling of `max_speed_mps` in small increments from the Entry 3 baseline, validating on all 5 seeds after each change (see the guidance given directly to the team for this).
- If returning to automated search, use 3+ training seeds instead of 2, and/or increase the damage penalty scale further, before trusting a search result over Entry 3's baseline.
- Consider the lookahead-driven anticipatory apex bias again later — the idea itself was not disproven, it was just tested simultaneously with a large, uncontrolled speed increase, which makes it impossible to tell which change caused the damage.

Entry 5 — Neuroevolution: Five Training Seeds at a Scaled-Up Search Budget

Date and time: August 31, 2026 (continued)

Participants and contributions:

- Jayden Webb — proposed widening Entry 2's training seed count from two to five to directly target the cross-seed generalization failure, directed the sequencing of the experiment (change seed count first, observe, only then scale population/generations rather than changing both at once), requested held-out validation against the actual suite before trusting the training curve, watched the resulting genome race live, and directed the genome swap and this documentation update.
- AI coding agent (Claude Code) — implemented the seed-count change in `scripts/train_neuroevolution.py`, ran both training passes (unscaled and scaled budget), diagnosed why the first regressed, wrote a standalone held-out validation script, ran the graphical head-to-head to confirm the result visually, and performed the `BEST_GENOME` swap plus this entry and the `EXPLORATION.md` update.

Question or objective: Entry 2 showed that training on more seeds (attempt 4, two seeds) made results *worse*, not better, collapsing to the same degenerate idle genome as the single-objective survival-only attempt. Was that because more training seeds are fundamentally the wrong lever, or because the population/generation budget (10 genomes, 6–8 generations) was already too small for two seeds and got asked to solve an even harder problem without more capacity? Concretely: does five-seed training, given a search budget scaled to match, produce a genome that survives the *held-out* suite — the specific failure every prior attempt shared — without giving up neuroevolution's speed advantage?

What we investigated or changed:

- Changed `scripts/train_neuroevolution.py`'s `--seeds` default from `[13, 55]` to `[13, 55, 7, 89, 233]` — five seeds, still kept distinct from the held-out validation suite `(42, 110, 271, 997, 2027)`.
- Ran training at the unchanged default budget (population 10, generations 6) first, deliberately, to isolate the effect of the seed-count change alone before touching anything else: **best training fitness converged to only 8.8 m**, with the `worst`-of-generation column pinned near the full –350 elimination penalty in every generation — confirming the same population could not satisfy five seeds' worth of requirement at all, let alone generalize.
- Reran with population and generations both scaled up (10→20, 6→15) to match the harder five-seed objective: **best training fitness converged to 235.8 m** by generation 9 and held there. The `worst`

column still touched -350 in the final generation, showing the population was not uniformly safe even though its best genome was.

- Validated the resulting genome on the full held-out suite with a standalone script (not the shipped `controllers.neuro`, to avoid touching `BEST_GENOME` before the result was confirmed): `run_headless_head_to_head` against a passive baseline, all five held-out seeds, five races each (25 total), 30-second rounds — identical protocol to every other attempt.
- Watched the validated genome race live (`uv run racing h2h --watch`) against `controllers.reactive` (seed 110) to visually confirm the driving behavior before committing to the swap.
- Swapped the new genome into `BEST_GENOME` in `src/controllers/neuro.py` and re-ran `scripts/evaluate_controller.py --module controllers.neuro` on a held-out seed to confirm the shipped controller now reproduces the validation script's numbers exactly.

Evidence:

- Sources or documentation: none new; built on Entry 2's fitness-function design (unchanged three-tier scoring) and Entry 3/4's lesson about training-seed coverage vs. search budget.
- AI-agent assistance: Claude Code ran both training passes in the background, read the raw generation-by-generation output to diagnose the first pass's failure before proposing the budget increase, and wrote/ran the held-out validation independently of the training script so the result wasn't self-reported by the same code path that produced it.
- Commits or code: `scripts/train_neuroevolution.py` (seed default), `src/controllers/neuro.py` (`BEST_GENOME` replaced).
- Experiment output: `uv run python scripts/train_neuroevolution.py` (unscaled budget, regressed) and `uv run python scripts/train_neuroevolution.py --population 20 --generations 15` (scaled budget, converged); held-out validation via `run_headless_head_to_head` across (42, 110, 271, 997, 2027), 5 races/seed; confirmation run via `uv run python scripts/evaluate_controller.py --module controllers.neuro --seed 42 --races 2 --round-seconds 30`.
- Leaderboard result: not applicable — local evaluation only.

What we observed:

- Widening the training objective (more seeds) and widening the search's capacity to solve it (more population/generations) are not independent levers — increasing seed count alone, at Entry 2's original budget, made the outcome strictly worse (8.8 m vs. attempt 4's already-poor 53 m), reproducing the same kind of collapse for a harder reason.
- Once budget was scaled to match, the held-out result was the best of any neuroevolution attempt so far: **24/25 survived, 25/25 completed ≥ 1 lap, avg scored distance 356.0 m, max speed 28.9 m/s** — compared to attempt 3's 8/25 survived, 21/25 laps, ~187–288 m avg.
- The one held-out failure (seed 997, race 4: eliminated, damage 1.00) shows the fix is not complete — the fitness function still scores a genome by the *mean* across its five training seeds, so a genome that is excellent on four and fails on one can still average well. This matches the exact risk discussed before running the experiment.
- This result closes most, but not all, of the gap to Approach 1 (25/25 survived, 435 m) while keeping neuroevolution's $\sim 1.8\times$ top-speed advantage (28.9 m/s vs. 15.7 m/s), materially changing the recommendation in `EXPLORATION.md`'s comparison section from "Approach 1 dominates" to "Approach 1 is safer, Approach 2 is a live candidate."

Decision and rationale: Replaced attempt 3's genome with this one as `BEST_GENOME` in `src/controllers/neuro.py`, and updated `EXPLORATION.md`'s Approach 2 section and comparison table to reflect the new numbers. This is a clear improvement on every held-out metric simultaneously (survival, laps, and distance all rose; speed did not have to be sacrificed to get there), unlike prior attempts which always traded one property for another.

Next steps:

- The remaining single held-out elimination suggests the mean-across-seeds fitness is still the limiting factor, not search budget — try scoring by the worst-of-seeds result (or a mean-minus-spread penalty) instead of the plain mean, now that the budget is large enough to make that harder objective tractable.
- Consider increasing training seeds beyond five and/or adding more elites, now that a population-20/generation-15 run completes in well under 15 minutes.
- Re-run Approach 1 vs. Approach 2 selection reasoning now that the gap has narrowed substantially — this may no longer be a clear-cut "pick Approach 1" decision by the time of the "Select and Refine" stage.

Entry 6 — Reactive Controller: Proactive Corner Speed Control and Quadratic Braking (Manual A/B)

Date and time: August 31, 2026 (continued)

Participants and contributions:

- Leon Aaron — directed the experiment loop (inspect → modify → optimize → validate → diagnose), specified the train/validation seed split, and required one-variable-at-a-time testing before trusting any automated search result.
- AI coding agent (Claude Code) — added the new tunable mechanisms, ran every isolated A/B test, diagnosed the counterintuitive results, and added parameter bounds plus a two-phase search to `scripts/optimize_reactive.py`.

Question or objective: Two hypotheses to test in isolation before touching anything else: (1) does proactively reducing target speed for an anticipated turn (`heading_error_degrees`) or an actual one already underway (IMU `yaw_rate_degrees_per_s`) let the car go faster overall by avoiding hard reactive braking later? (2) does a speed-squared braking-distance term (matching true constant-deceleration stopping distance, instead of the current linear-in-speed formula) improve safety margin at high speed?

What we investigated or changed:

- Redefined the seed split per instruction: training (`13, 55, 110, 271, 997`), validation (`42, 2027, 8675, 31415, 777001`) — the last three are seeds never used anywhere in this project before, for a genuinely unseen check.
- Added two new, independently-toggable mechanisms to `ReactiveParams/_throttle_command` in `src/controllers/reactive.py`, both defaulting to zero gain (no effect) so each could be A/B tested without disturbing the existing Entry 3 baseline: `corner_speed_gain` (+ `corner_signal_deg`, `corner_yaw_rate_deg_per_s`) for proactive slowdown, and `brake_quadratic_coeff` added to the existing linear brake-distance formula.
- Confirmed the refactor was behavior-preserving at gain=0: re-ran the training-seed suite and reproduced the exact prior baseline, 432.5 m avg, 25/25 survived (one pre-existing 0.03-damage graze

on seed 13's 5th starting position, noted but not chased further — trivial and non-blocking).

- Tested each mechanism alone at the existing speed (`max_speed_mps=15.8`): `corner_speed_gain=0.3` → **260.7 m** (−40%); `brake_quadratic_coeff=0.02` → **353.8 m** (−18%). Both pure regressions, zero damage in both cases.
- Tested `max_speed_mps=19` alone, and combined with each mechanism, to see whether either compensates for a higher speed target rather than just adding caution to an already-safe baseline: alone → **395.1 m** (small damage on seed 13); + `corner_speed_gain=0.3` → **295.0 m**; + `brake_quadratic_coeff=0.03` → **325.4 m**. All three are worse than the 432.5 m baseline.

Evidence:

- AI-agent assistance: Claude Code implemented both mechanisms behind a zero-gain default specifically so a byte-for-byte baseline reproduction check was possible before any A/B test, then ran all five isolated trials via `scripts/try_params.py --seeds 13 55 110 271 997 --<flag> <value>`.
- Commits or code: `src/controllers/reactive.py` (new tunable throttle mechanisms), `scripts/try_params.py` (`--seeds` override added), `scripts/optimize_reactive.py` (`PARAM_BOUNDS` + clamped mutation + two-phase search support).
- Experiment output: six `scripts/try_params.py` runs (25 races each) as listed above.
- Leaderboard result: not applicable.

What we observed:

- Both new mechanisms are pure regressions in isolation, because the baseline already achieves 0.00 damage everywhere — there is no existing safety gap for extra caution to fill, so any additional slowdown or braking margin only costs distance.
- Raising `max_speed_mps` alone is *also* a regression (395 m vs. 432.5 m), which is counterintuitive until traced: the brake-distance formula scales with the car's *actual* current speed, so a car that actually reaches a higher top speed on the straight also demands a larger stopping distance before the next corner at the *same* wall-lidar reading — triggering earlier, harder braking that costs more than the higher top speed gains. This is a real parameter interaction, not a bug.
- Adding either safety mechanism on top of the higher speed did not fix this — both remained worse than the untouched 15.8 m/s baseline. This confirms Entry 3/4's parameters are a genuinely co-tuned joint optimum: every single-variable change tested made things worse, which is exactly the signature of a well-balanced local optimum rather than a poorly-tuned one.

Decision and rationale: Concluded that manual single-variable A/B testing has reached its limit here — the remaining potential improvement, if any, requires jointly re-tuning multiple parameters together (e.g. speed *and* brake formula *and* apex bias simultaneously), which is exactly the automated search's job, not a human's, once bounds exist to keep it from wasting evaluations. Did not accept any manual change; `DEFAULT_PARAMS` remains Entry 3's validated 432.5–435 m baseline pending the search result below.

Next steps:

- Run the newly bounded, two-phase (broad exploration then fine-tuning) search on all five training seeds at once — directly targeting the cross-seed generalization failure from Entry 4, which only ever trained on two.
- Validate any search result against the full, disjoint validation set (42, 2027, 8675, 31415, 777001) before accepting it, exactly as required this time.

- If the search cannot beat 432.5 m either, that is itself a meaningful, reportable conclusion: this parameterization of the reactive controller may be near its practical ceiling on this track without a further architectural change (e.g. the anticipatory lookahead-based apex bias from Entry 4, tested in isolation this time rather than simultaneously with a speed increase).